Answer Sheet

Write your answers on this sheet. Exam questions & instructions can be found on subsequent sheets.

Name:

Student ID number:

Answers to knowledge questions:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A																				
B																				
C																				
D																				

Answers to hands-on questions:

	21	22	23	24	25	26	27	28	29	30
A										
B										
C										
D										

31

valM	
R[rsp]	
R[rA]	
PC	

32

VPN	
TLB index	
TLB tag	
TLB hit	
Page fault	
PPN	

33

A	
B	

34

	SU	ST	PU	PT
SU				
ST				

Answers to analytical questions:

35

36

End of answer sheet.

This page was intentionally left blank.

Exam

Willard Rafnsson
IT University of Copenhagen

January 10, 2025

Course name: Operating Systems and C

Course codes: BSOPSYC1KU and KSOPSYC1KU

Aids allowed: Written and printed books and notes. Pen.

Duration: 5 hours.

Write all of your answers on the provided answer sheet.

The grade is determined based on an overall assessment of all answers to the exam questions.

A blank answer yields 0 points. A correct answer yields a fraction of the available points in the question. An incorrect answer yields negative points, smaller the more available answer options there are.

For each option in multiple-choice questions, write **T** or **F** (true or false) in the appropriate field on the answer sheet. A multiple-choice question can have 1 to 4 correct options.

Do not hand in pages 1 to 12; we will not read them.

Good luck.

Knowledge Questions

Question 1

Where does the first machine code instruction that your computer executes, upon being powered on, reside?

A. RAM
B. ROM on the motherboard
C. CPU
D. boot loader on disk

Question 2

What is the job of a boot loader?

A. power-on self-test.
B. load the OS kernel image into RAM.
C. pass control to the OS initialization code.
D. start the first process.

Question 3

Which of the following does the CPU use to determine what code to execute, upon receiving an interrupt?

A. PC
B. IRQ
C. IDTR
D. interrupt number

Question 4

What causes a running process to eventually yield control, such that another process can run in its place?

A. the running process itself.
B. the process that will run in its place.
C. scheduler.
D. timer interrupt.

Question 5

What must take place during a context switch?

A. an interrupt handler is executed.
B. process-specific register values are saved to disk.
C. scheduler chooses which process executes next.
D. process is put into the "blocked" state.

Question 6

Rule 3 of Multi-Level Feedback Queues states that when a job enters the system, it gets top priority. Now consider the following edge case: P is a program that creates a new process that also executes P, then orphans the new process, and then terminates. Suppose P is executed, and that the resulting process is the sole process with topmost priority on the system. Which of the following rules prevent starvation of lower-priority processes?

A. rule 1 (if priority(A) > priority(B), then A is scheduled).

B. rule 2 (if priority(A) = priority(B), A and B are scheduled in round-robin fashion).

C. rule 4 (when job uses its CPU time allotment, move job to lower queue).

D. rule 5 (after time S, move all jobs to topmost queue).

Question 7

What prevents a process running in user-mode from writing to kernel-space memory?

A. monitor.

B. supervisor bit (in page table entry).

C. current privilege level (in the CPU).

D. MMU.

Question 8

What prevents a process running in user-mode from entering kernel-mode without passing control to kernel-space code?

A. instruction set.

B. monitor.

C. namespaces.

D. virtual memory.

Question 9

Which of the following statements about file systems are true?

A. a file can have zero links to it.

B. a file can have more than one link to it.

C. the file system keeps track, for each process, which files it is using.

D. the process keeps track of its cursors in each file it is using.

Question 10

How does a multicore CPU execute a parallel program?

A. instruction set lets the programmer specify on which core an instruction should run.

B. one core directly signals another core to start running a task.

C. a task is created by means of a system call.

D. cores pick tasks from a shared pool of tasks in memory.

Question 11

Each thread has its own...

A. stack
B. heap
C. global values
D. text data

Question 12

Which one of the following addresses is 8-byte aligned?

A. `0xEDE7`
B. `0xEDE0`
C. `0xEDE4`
D. `0xEDE6`

Question 13

Consider a 4-way set associative cache (E = 4). Which one of the following statements it true?

A. The cache has 4 blocks per line.
B. The cache has 4 sets per line.
C. The cache has 4 lines per set.
D. The cache has 4 sets per block.

Question 14

Which of the following can be utilized to eliminate race conditions?

A. lock instruction prefix.
B. cache coherence protocol.
C. bus arbiter.
D. compare and swap.

Question 15

Which of the following are valid uses of memory barriers?

A. protect process memory from being accessed by other processes.
B. prevent instruction reordering.
C. lock-free programming.
D. containers.

Question 16

Which of the following are benefits of register renaming?

- **A.** eliminates false data dependencies.
- **B.** reveals more instruction-level parallelism.
- **C.** facilitates out-of-order execution.
- **D.** helps better utilize the resources of a superscalar processor.

Question 17

Which of the following statements regarding memory aliasing are true?

- **A.** memory aliasing can prevent the compiler from optimizing away some memory I/O.
- **B.** you must under no circumstance use two different pointers that can specify the same location.
- **C.** introduce local variables to eliminate the possibility of aliasing.
- **D.** keep pointers in separate scopes to prevent the possibility of aliasing.

Question 18

Why is software prefetching generally discouraged today?

- **A.** it costs an instruction.
- **B.** you might negatively impact performance.
- **C.** your code performance will be less portable.
- **D.** software prefetching is error-prone.

Question 19

How are Linux Security Modules implemented in Linux?

- **A.** by means of access control lists (ACLs).
- **B.** a security check is made before any security-sensitive operation.
- **C.** with namespaces and cgroups.
- **D.** hooks invoke the LSM kernel module to resolve security check.

Question 20

When creating a container, what are namespaces used for?

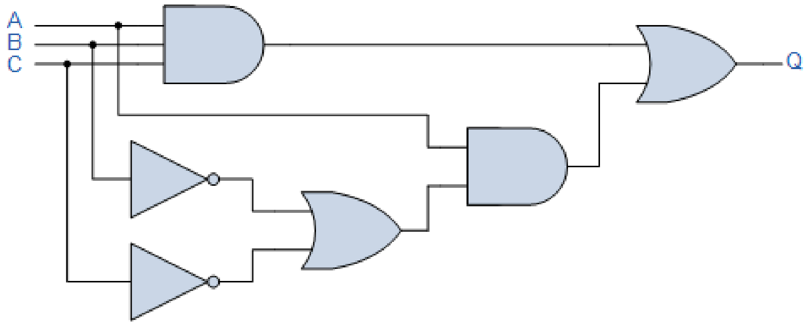
- **A.** isolation
- **B.** provisioning
- **C.** virtualization
- **D.** hardening

End of knowledge questions. Exam continues on the next page.

Hands-on Questions

Question 21

Here is a logic diagram.



(The inputs of the circuit, top-to-bottom, are A, B and C. The output is Q.)

Which of the following Boolean expression are equivalent with Q (the output of the circuit)?

A. `A&B|~C`

B. `A&(~C|~B) | C&B&A`

C. `A&( B&C | ~(B | C) )`

D. `A`

Question 22

Consider the following code.

```
int x[2][128];
int i;
int sum = 0;
for ( i = 0 ; i < 128 ; i++ ){
  sum += x[0][i] * x[1][i];
}
```

Assume we execute this under the following conditions.

- `sizeof(int)` = 4.
- array `x` begins at memory address `0x0` and is stored in row-major order.
- the cache is initially empty.
- the only memory accesses are to the entries of array `x`; all other variables are stored in registers.

Assume that the cache is 512 bytes, direct-mapped, with 16-byte cache blocks. What is the miss rate?

A. 0%

B. 10%

C. 50%

D. 100%

Question 23

Consider an image processing scenario, where each image is of size `h * w * c`, where `h` is the image height, `w` its width, and `c` the number of channels (i.e. spectral bands) in the image.

(The image could have the typical three spectral bands (red, green and blue), or it could be a multi-spectral image with tens, even hundreds of spectral bands, as is the case in e.g. deep-space imaging).

Suppose you wish to scale each channel `k` by a constant `sca[k]`, and add the resulting channels together, producing the single-channel image `out`. Here is a code sketch that achieves this.

```
for ( int i = 0; i < h; i++ ) {
  for ( int j = 0; j < w; j++ ) {
    double o = 0.0;
    for ( int k = 0; k < c; k++ ) {
      o += img[ ??? ] * sca[ k ];
    }
    out [ i, j ] = o;
  }
}
```

We have left unspecified how `img` is indexed (indicated by the `???`).

How should `img` be stored to maximize cache efficiency?

A. `h*w*c`

B. `w*h*c`

C. `c*h*w`

D. `c*w*h`

Question 24

Now suppose you use GPU processing with CUDA to improve the performance of the image scaler from before. After computing `i` and `j` (using `threadIdx`, `blockIdx`, and `blockDim`), the remainder of the body of the CUDA kernel is as follows.

```
double o = 0.0;
for ( int k = 0; k < c; k++ ) {
  o += img[ ??? ] * sca[ k ];
}
out [ i, j ] = o;
```

Again we have left unspecified how `img` is indexed (indicated by the `???`).

How should `img` now be stored to maximize cache efficiency?

A. `h*w*c`

B. `w*h*c`

C. `c*h*w`

D. `c*w*h`

Question 25

What kind of locality does the code snippet in **Question 23** exhibit?

A. spatial locality of data with respect to elements of `sca`.

B. temporal locality of data with respect to elements of `sca`.

C. spatial locality of instructions with respect to the innermost loop body.

D. temporal locality of instructions with respect to the innermost loop body.

Question 26

Consider the following C source file `t.c`.

```
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
int s = 0;
void *add(void* arg) {
  int a = s + (int)arg;
  usleep( ( (float)rand() / RAND_MAX ) * 10 );
  s = a;
}
int main() {
  pthread_t tid2;
  pthread_t tid3;
  srand(time(NULL));
  pthread_create( &tid2, NULL, add, (void*)2 );
  pthread_create( &tid3, NULL, add, (void*)3 );
  pthread_join(tid2, NULL); // thread 2 join
  pthread_join(tid3, NULL);
  printf("%d\n", s);
}
```

It compiles (with warnings) with the following command

```
gcc -pthread -o t t.c
```

Upon executing this program (`./t`), which of the following are possible outputs of the program?

A. 0

B. 2

C. 5

D. 10

Question 27

Which of the following statements about the code snippet from **Question 26** are true?

A. the program can enter a livelock.

B. protecting access to `s` by a mutex will prevent race conditions.

C. moving the line marked "thread 2 join" 1 line up will prevent race conditions.

D. the critical region consists of all instructions that can be encountered after thread creation.

Question 28

We consider code that adds numbers in an array in a particular way. Assume the following declarations.

```
unsigned short n;
unsigned short num[n];
unsigned long sum;
```

The num array is filled with numbers (e.g. from standard input). Finally, the following "sum code" is run on the array.

```
// sum code
for (unsigned short i = 0; i < n; i++) {
  if (num[i] >= 256)
    sum += num[i];
  }
}
```

This sum code runs faster if num is sorted. Why is that?

(On a particular machine, for n = 32768, the sum code took 115 microseconds to run with num filled with random numbers, and 19.3 microseconds with num sorted).

A. branch prediction.

B. caching.

C. alignment.

D. code motion.

Question 29

Here is a short program written in Y86-64.

```
0   irmovq $0xFF, %rax
1   xorq %rdi, %rax
2   je i
3   rrmovq %rdi, %rax
4   halt
5 i:irmovq $2, %rax
6   addq %rdi, %rax
7   halt
```

Assume %rax = 0xcafe, and that the program is running on PIPE hardware (including forwarding- and pipeline-control-logic).

Hint: It may be helpful to simulate (on paper) this program running on the above-stated hardware, i.e. to fill out a figure along the following.

	pipeline				
Cycle	F	D	E	M	W
0	0				
1	1	0			
2	2	1	0		
3	...				

The program has one or more hazards in it. In which cycle does the pipeline encounter its first hazard?

A. cycle 3

B. cycle 4

C. cycle 5

D. cycle 6

Question 30

What kind of hazard did the program in **Question 29** encounter first?

A. load/use hazard

B. read-after-write

C. branch misprediction

D. ret

Question 31

Consider the following Y86-64 instruction.

```
POPQ %rdi
```

Show how this instruction executes, by filling in values in fields **valM**, **R[rsp]**, **R[rA]**, and **PC**.

Assume SEQ hardware. At the time this instruction is executed, `%rdi` has value `0x42`, `%rsp` has value `0x62fe`, `PC` has value `0x20241e`, and the words stored in memory near `%rsp` are as follows.

```
M8[ 0x62f0 ] = 0xabacadabacab0000
M8[ 0x62f8 ] = 0x0badf00dcafe0000
M8[ 0x6300 ] = 0x0000feedfacebeef
```

The values you fill into the fields should be in hexadecimal notation. Please leave out the leading `0x`.

Question 32

Consider the following memory system.

- page size is 256 bytes.
- virtual addresses are 14 bits long.
- physical addresses are 12 bits long.
- there is a 3-way TLB with 4 sets.
- memory is byte-addressable.

The memory system thus has the following system parameters: **VPO** = 8, **PPO** = 8, **VPN** = 6, **PPN** = 4.

Suppose the page table contains the following data.

VPN	PPN	Valid	VPN	PPN	Valid
10	7	1	28	b	1
11	6	1	29	8	0
12	1	1	2a	f	1
13	3	1	2b	d	0
14	2	1	2c	e	1
15	4	0	2d	a	0
16	5	0	2e	9	1
17	0	0	2f	c	0

Suppose the TLB contains the following data.

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	4	7	1	5	2	1	a	b	1
1	4	6	1	5	4	0	a	8	0
2	4	1	1	5	5	0	a	f	1
3	4	3	1	5	0	0	a	d	0

Show how the virtual address `0x16fa` is translated to a physical address, by computing **VPN**, **TLB index**, **TLB tag**, **TLB hit**, **Page fault**, and **PPN**.

Provide values in hexademical notation (leaving out the leading `0x`), except where a true/false answer is requested (i.e. in the case of **TLB hit** and **Page fault**); provide **T/F** as an answer there instead. Write **NA** to indicate that there is no value to provide.

Question 33

Consider a program that takes 10 minutes to run on 4 processors, and which has an estimated parallel fraction F = 0.8.

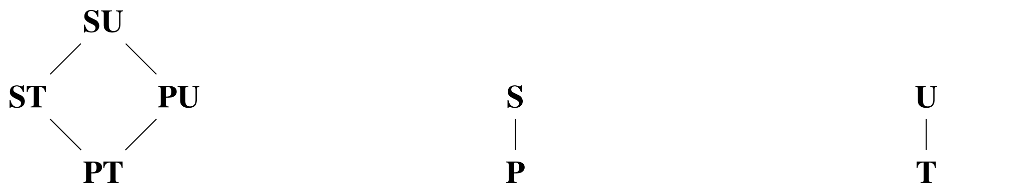
A. How many minutes does this program take to run on 1 processor?

Suppose the execution time of the non-parallelizable part of the workload can be reduced to 3 minutes.

B. How many minutes does such a version of the program take to run on 4 processors?

Question 34

Consider the following three lattices of security levels.



product lattice confidentiality lattice integrity lattice

A security lattice states that data of a given security level can only flow to a storage which has a security level at or above the security level of the data. The confidentiality lattice, with its secret (**S**) and public (**P**) security levels, thus states that data must be protected from undesired disclosure. The integrity lattice, with its untrusted (**U**) and trusted (**T**) security levels, states that data must be protected from undesired destruction. The product lattice states both at the same time. For instance, a file labeled **PU** contains public data that is trusted (e.g. that we are sharing, but wish to protect from corruption by untrusted data).

Here we consider a Mandatory Protection System (MPS) based on labels drawn from this product lattice. Fill out (on the answer sheet) the first two rows of the access matrix for this MPS. Each field can either be blank (indicated by `-`), `r` , `w` , or `rw`.

		file			
		SU	**ST**	**PU**	**PT**
process	**SU**				
	ST				
	PU				
	PT				

End of hands-on questions.

Analytical Questions

Question 35

Why is it advised to strive for a stride-1 memory access pattern in your code?

Provide an answer on the answer sheet. We expect up to three sentences.

Question 36

How does “fetch-and-add” help us implement mutexes?

Provide an answer on the answer sheet. We expect up to three sentences.

End of analytical questions.

End of exam.